R1 Arm-Hand MuJoCo Training

Cesar A. Pascal Vasquez

October 2026

Abstract

This paper presents a simulation-based manipulation study in which the right arm of a Unitree R1 humanoid is integrated in MuJoCo with a BrainCo Revo 2 Basic right hand. The R1 pelvis is fixed to isolate upper-limb manipulation, and the original terminal hand geometry is replaced by the Revo 2 model through a virtual wrist sleeve/flange. The task requires the system to reach toward a ball, establish a power grasp, retain the ball during a lift and windup, and release it toward a randomized target. Control is provided by a 51–128–128–11 feedforward neural network that predicts residual corrections around a safe nominal reach–grasp–throw trajectory. Training uses domain-randomized behavioral cloning from 240 scripted inverse-kinematics teacher episodes followed by two rounds of Dataset Aggregation (DAgger). The final aggregated dataset contains 63,985 state–action samples. Evaluation over 100 previously unseen randomized MuJoCo episodes produces an 85% complete-task success rate, with 99% reach, 96% grasp, 91% hold, 91% release, and 85% throw success. Mean minimum reach error is 13.9 mm, and mean forward throw distance is 0.209 m. Compared with the earlier hand-tuned simulated controller, the learned residual policy improved full-sequence reliability while maintaining strong performance across the individual task stages. This work is simulation-only and does not include physical-robot validation. The virtual wrist adapter is an engineering approximation and must be physically calibrated and safety-validated before sim-to-real deployment.

I. Introduction

Humanoid manipulation requires coordinated control of gross arm motion and contact-rich hand behavior. Even for a short sequential task, successful execution depends on several interacting elements: the arm must approach an object without excessive disturbance, the fingers must form a stable grasp, the object must remain retained while the arm changes configuration, and the release must occur with appropriate timing and velocity. These challenges become more pronounced when object properties, target position, actuator response, sensing, and command timing vary between trials.

This work studies that problem entirely in the MuJoCo physics simulator [1]. The simulated system combines the right arm of a Unitree R1 humanoid with a BrainCo Revo 2 Basic right hand. The integration uses robot-description and simulation resources consistent with the Unitree MuJoCo software ecosystem [3], [4] and the Revo 2 documentation and digital assets [5]. The R1 pelvis is fixed so that the experiment focuses on arm–hand coordination rather than whole-body balance.

The primary task is a reach–grasp–hold–throw sequence. Rather than requiring a neural network to discover the complete motion from scratch, the controller is structured as a residual policy around a safe nominal trajectory. A scripted inverse-kinematics teacher supplies reach, lift, windup, and throw waypoints together with a predefined power grasp. Behavioral cloning provides the initial policy, and two rounds of Dataset Aggregation (DAgger) [2] add corrective labels for states actually visited by the learned policy. Domain randomization is used throughout data collection so that the policy must respond to variation in ball placement, target location, ball dynamics, actuator gains, control delay, and noisy observations.

We use this setup to study integrated arm–hand control under a defined randomized simulation task. The paper documents the model integration, observation and action spaces, residual-policy formulation, training procedure, randomization ranges, evaluation criteria, and final performance. We also compare the learned controller with an earlier hand-tuned simulated controller.

II. Integrated Simulation Model

A. Robot and Hand Integration

The simulated robot is a Unitree R1 humanoid with a BrainCo Revo 2 Basic right hand mounted at the distal end of the R1 right arm. The original R1 terminal hand geometry is removed from the model. A virtual sleeve/flange connects the Revo 2 hand model to the R1 right wrist and defines the kinematic interface used in the simulation.

The R1 pelvis is fixed in the MuJoCo world. All non-task R1 joints are maintained in a standing posture, which removes balance as a confounding factor and permits the study to focus on upper-limb manipulation. The controlled R1 arm joints are shoulder pitch, shoulder roll, shoulder yaw, elbow, and wrist roll. The six active Revo 2 commands correspond to the thumb metacarpal, thumb proximal, index proximal, middle proximal, ring proximal, and pinky proximal joints. Mechanically coupled or passive hand motion follows the hand model rather than receiving separate policy outputs.

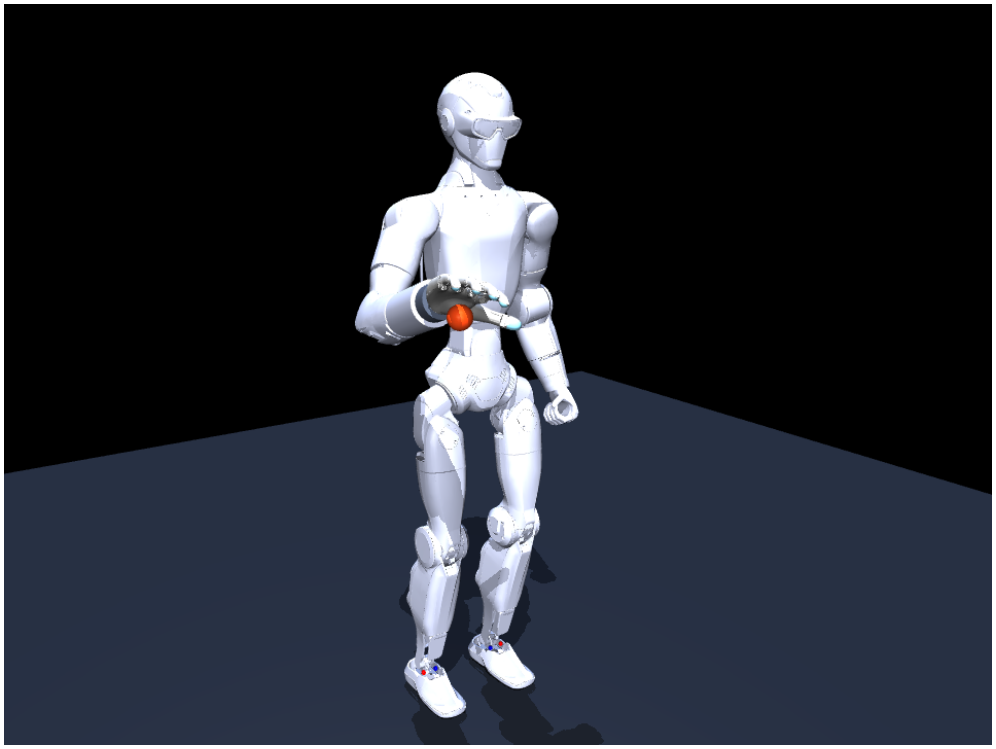


Fig. 1. Integrated R1–Revo 2 MuJoCo model. The pelvis is fixed to isolate arm–hand manipulation.

B. Task Sequence

Each episode runs for at most 5.30 s with a 40 Hz control frequency, corresponding to a maximum of approximately 212 control steps. The task is separated into time-indexed phases summarized in Table I. The timing defines the nominal structure of the motion while the residual policy provides state-dependent corrections.

TABLE I
NOMINAL TASK TIMING.

Phase	**Nominal time**	**Purpose**
Reach	0.00–1.45 s	Move open hand toward the ball
Hand closing	≈1.45–3.00 s	Form a power grasp
Lift	3.00–3.45 s	Raise the grasped ball
Windup	3.45–3.80 s	Move arm backward in preparation
Release	3.88 s	Open the hand to release the ball
Throw completion	4.30 s	Complete the forward throw motion
Episode limit	5.30 s	Maximum simulated duration

During the reach phase, the hand remains open while the arm tracks a reachable configuration near the ball. Figure 2 shows the policy approaching the object before the grasp is established.

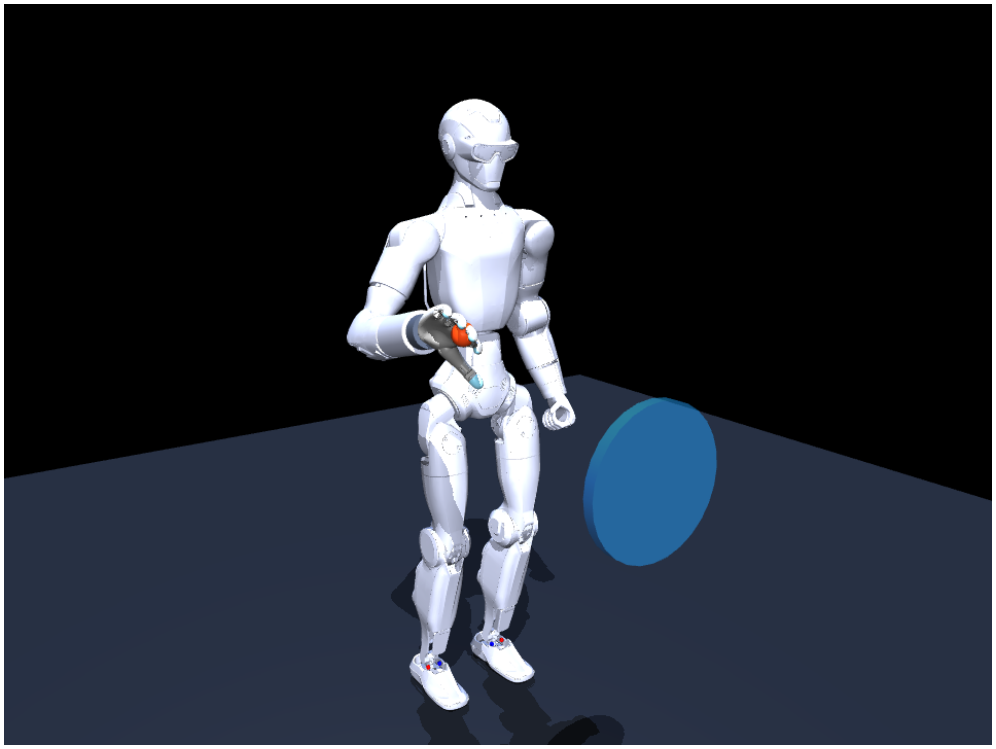


Fig. 2. Reach phase of the trained policy.

During hand closing, the fingers form a power grasp around the ball. The ball initially uses a temporary fixture to stabilize the setup; as the grasp stage proceeds, the fixture is released so that hand–ball contact and object dynamics determine whether the grasp is maintained. Figure 3 shows the power-grasp stage.

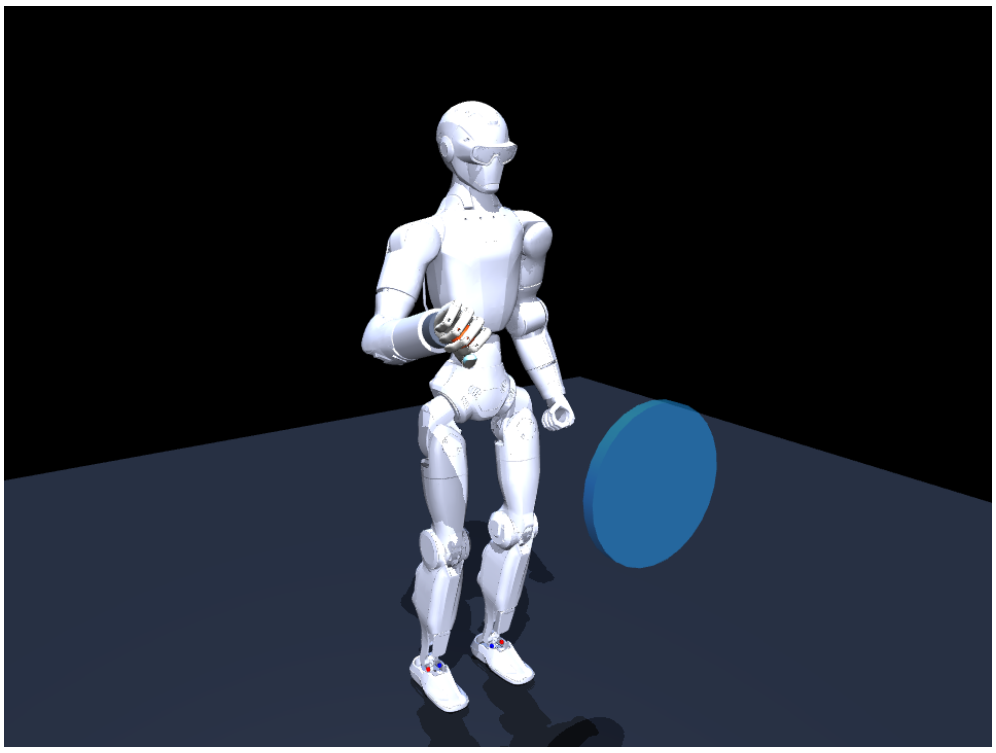


Fig. 3. Power-grasp phase.

After grasp formation, the arm lifts the ball and moves through the windup configuration. The forward throw then begins,

and the hand opens at 3.88 s to release the object toward the target. Figure 4 shows the release/throw stage.

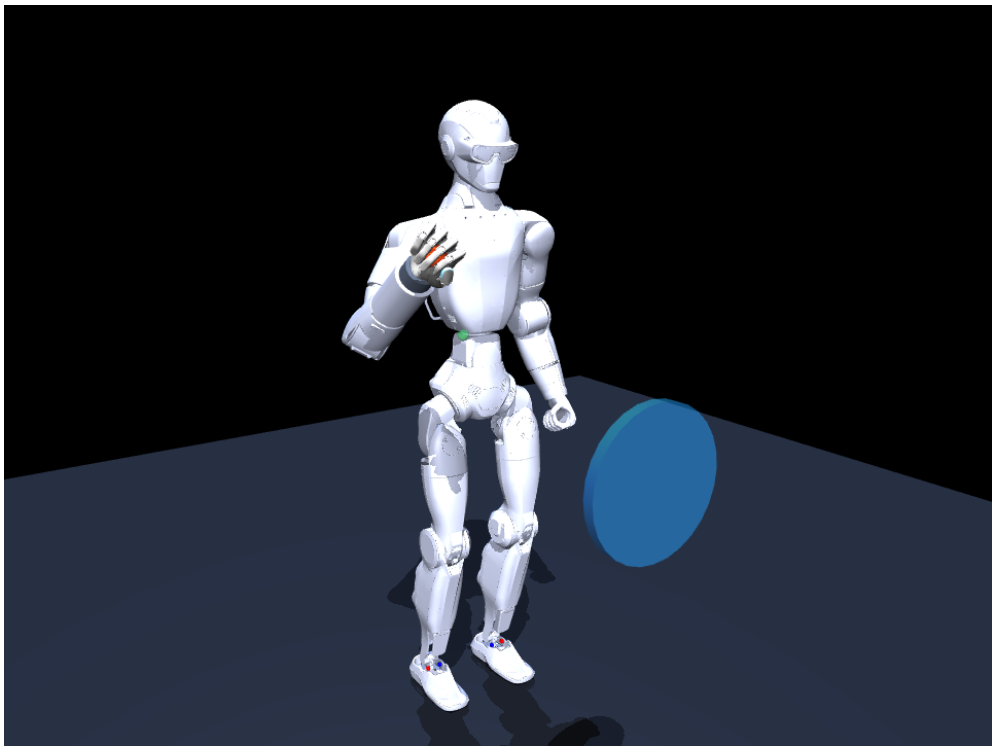


Fig. 4. Release and throw phase of the trained policy.

III. Methods

A. Observation Space

At each control step, the policy receives a 51-element observation vector. The observation includes normalized time, a task-phase representation, relative ball and target positions, palm position and velocity, ball velocity, arm joint positions and velocities, hand joint positions, hand–ball contact state, fixture state, and the previous action. These quantities provide both task context and short-term control context without requiring image-based perception.

A compact representation is

$$\mathbf{o}_t \in \mathbb{R}^{51}, \tag{1}$$

where the components combine task timing, relative geometry, robot state, object dynamics, contact information, and the preceding command. The use of relative ball and target positions allows the same policy to respond to randomized placements rather than memorizing one world-frame trajectory.

B. Action Space

The action vector contains 11 normalized joint commands,

$$\mathbf{a}_t \in [-1, 1]^{11}, \tag{2}$$

consisting of five R1 arm commands and six Revo 2 hand commands. Each normalized component is mapped to the corresponding allowable joint target in the simulator. The arm outputs control shoulder pitch, shoulder roll, shoulder yaw, elbow, and wrist roll. The hand outputs control the six active Revo 2 joints used in this study.

C. Residual Policy

The learned controller is a feedforward neural network with architecture

$$51 \to 128 \to 128 \to 11, \tag{3}$$

with hyperbolic tangent activations. The network does not generate the complete manipulation trajectory independently. Instead, it predicts corrections to a nominal reach–grasp–lift–windup–throw sequence.

Conceptually, the commanded normalized action can be written as

$$\mathbf{a}_t = \mathrm{clip}(\mathbf{a}_t^{\mathrm{nom}} + \Delta\mathbf{a}_\theta(\mathbf{o}_t), -1, 1)\,, \tag{4}$$

where $\mathbf{a}_t^{\mathrm{nom}}$ denotes the safe nominal action and $\Delta\mathbf{a}_\theta$ is the neural residual. Equation (4) describes the controller structure; the residual is interpreted as a correction in the normalized command space. The nominal trajectory handles the broad sequencing of the task, whereas the network adapts the motion to randomized ball position, target position, contact evolution, actuator response, and observation disturbances.

This design reduces the learning burden relative to learning a full throwing motion from an unconstrained initial policy. It also preserves a meaningful default motion when the learned correction is small.

D. Teacher Controller

The teacher is scripted rather than learned. It uses inverse-kinematic arm waypoints for the reach, lift, windup, and throw portions of the task and combines those arm targets with a predefined power-grasp command for the Revo 2 hand. The teacher therefore provides a coherent nominal action for each state encountered during demonstration collection and DAgger relabeling.

The teacher is not treated as an optimal controller. Its role is to provide safe, task-directed labels and corrective actions from which the neural policy can imitate and generalize under randomized conditions.

IV. Training Procedure

A. Behavioral Cloning

Training begins with 240 randomized teacher episodes. From these demonstrations, state–action pairs are collected and used for supervised behavioral cloning. Let $\hat{\mathbf{a}}_t$ denote the teacher label and $\mathbf{a}_\theta(\mathbf{o}_t)$ the network prediction. The training loss is a weighted mean-squared error,

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{n=1}^{N}\sum_{j=1}^{11} w_j \left(a_{\theta,j}^{(n)} - \hat{a}_j^{(n)}\right)^2, \tag{5}$$

where the arm-output weights are larger than the hand-output weights. This weighting emphasizes precise upper-arm positioning because reach geometry and throwing motion are strongly influenced by the five arm commands.

The initial optimizer learning rate is 1.2×10^{-3}, and the batch size is 512. Because the release and throw stages occupy less time than the approach and grasp stages, samples are balanced across task phases during training. This prevents the longer phases from dominating the supervised loss.

B. DAgger Correction Rounds

Behavioral cloning can suffer from covariate shift: small action errors move the policy into states that were uncommon in the original demonstration data. DAgger addresses this by collecting states under the learned policy while continuing to request teacher labels for those visited states [2].

Two DAgger rounds are used. During each round, control is mixed between the teacher and the learned policy during data collection. In the first round, the teacher action is executed with 45% probability. In the second round, that probability is reduced to 15%, exposing the data collection process to more states induced by the learned controller. Regardless of which controller executes the step, the teacher continues to supply the corrective label that is stored with the visited state.

For DAgger retraining, the learning rate is reduced to 75% of the initial value,

$$\eta_{\mathrm{DAgger}} = 0.75(1.2 \times 10^{-3}) = 9.0 \times 10^{-4}. \tag{6}$$

After the two rounds, the complete dataset contains 63,985 state–action samples.

C. Training Summary

Table II summarizes the main policy and optimization settings.

TABLE II
POLICY AND TRAINING CONFIGURATION.

Item	Setting
Policy architecture	51–128–128–11 feedforward network
Activation	$\tanh$
Controlled outputs	5 arm + 6 hand commands
Initial demonstrations	240 teacher episodes
Final dataset size	63,985 state–action samples
Training sequence	Behavioral cloning + 2 DAgger rounds
DAgger teacher-action probability	45% then 15%
Batch size	512
Initial learning rate	1.2×10^{-3}
DAgger learning rate	9.0×10^{-4}
Loss	Weighted mean-squared error
Sampling	Balanced across task phases

V. DOMAIN RANDOMIZATION

Randomization is applied so that the policy cannot rely on one exact object placement or one fixed set of simulated dynamics. Table III lists the ranges used for training and evaluation. The ball position varies by approximately ± 20 mm around its nominal placement. Ball radius, mass, and friction are randomized, as are actuator gains and command delay. Gaussian observation noise with standard deviation 0.010 is added, and the throw target is also randomized.

TABLE III
DOMAIN-RANDOMIZATION SETTINGS.

Randomized quantity	Range or setting
Ball position	Approximately ± 20 mm
Ball radius	16.5–19.5 mm
Ball mass	38–70 g
Ball friction	0.75–1.45
Actuator gains	82–118% of nominal
Command delay	0–2 control frames
Observation noise	Gaussian, $\sigma = 0.010$
Throw target position	Randomized

We apply these variations during data collection so that the policy must respond to changes in object placement, target placement, simulated dynamics, command timing, and observation noise rather than replaying one fixed trajectory.

VI. REWARD MONITORING

The simulation also computes a task-oriented reward that favors a small palm-to-ball distance, useful hand–ball contact, smooth control, and ball motion toward the target during the throwing stage, with a penalty when the ball falls below the working region. This reward is used to monitor behavior and summarize progress. The primary training signal for the reported policy, however, is supervised imitation of teacher actions through behavioral cloning and DAgger, not direct reward maximization.

VII. EXPERIMENTAL SETUP AND EVALUATION

A. Evaluation Protocol

The final neural policy is evaluated in 100 randomized MuJoCo episodes using seeds not used in the training episodes. No teacher actions are used to rescue the policy during this final evaluation. Each episode is scored using stage-specific criteria:

- **Reach success:** minimum hand-to-ball distance below 45 mm.
- **Grasp success:** at least two hand–ball contacts after fixture release.
- **Hold success:** ball retained through the windup stage.
- **Release success:** forward ball speed above 0.35 m/s.
- **Throw success:** forward ball travel above 0.16 m and minimum distance to the target below 0.50 m.
- **Complete success:** all required task stages succeed within the episode.

The reported continuous metrics are computed over the evaluation set and include minimum reach error, maximum hand–ball contacts, peak ball speed, peak forward speed, forward throw distance, minimum target distance, and episode duration.

VIII. Results

A. Final 100-Episode Evaluation

Table IV gives the final evaluation results. The learned policy completes the entire task in 85 of 100 unseen randomized episodes. Reach is the most reliable stage at 99%, followed by grasp at 96%. Hold and release both achieve 91%, and the final throw criterion is satisfied in 85% of episodes.

TABLE IV
Final policy results over 100 unseen randomized MuJoCo episodes.

Metric	Result
Complete task success	85%
Reach success	99%
Grasp success	96%
Hold success	91%
Release success	91%
Throw success	85%
Mean minimum reach error	13.9 mm
Mean maximum hand–ball contacts	9.1
Mean peak ball speed	2.95 m/s
Mean peak forward speed	0.606 m/s
Mean forward throw distance	0.209 m
Mean minimum target distance	0.260 m
Mean episode duration	4.56 s

The mean minimum reach error of 13.9 mm is well inside the 45 mm reach threshold. The mean maximum hand–ball contact count is 9.1, indicating that successful interaction generally involves multiple simultaneous or sequential contacts across the hand model. The mean peak ball speed is 2.95 m/s, while the mean peak speed in the forward target direction is 0.606 m/s. Mean forward throw distance is 0.209 m, and the mean minimum target distance is 0.260 m.

B. Comparison with the Earlier Hand-Tuned Controller

The final learned policy is compared with the earlier hand-tuned simulated controller in Table V. The baseline reported a 45% complete-task success rate, 72% reach success, 64% grasp success, 48% hold/release success, and a mean forward throw distance of 0.102 m. The final neural policy raises complete-task success to 85%, reach success to 99%, grasp success to 96%, and mean forward throw distance to 0.209 m.

TABLE V
Earlier hand-tuned controller versus final neural policy in simulation.

Metric	Earlier hand-tuned	Final neural policy
Complete task success	45%	85%
Reach success	72%	99%
Grasp success	64%	96%
Hold success	48%[a]	91%
Release success	48%[a]	91%
Mean forward throw distance	0.102 m	0.209 m

[a] The earlier project summary reported hold/release together at 48%; the same baseline value is shown in both rows only to align it with the two separately reported final-stage metrics.

The comparison shows a clear improvement in full-sequence reliability. The final policy also improves the reported reach and grasp stage rates and more than doubles the mean forward throw distance relative to the earlier hand-tuned controller. Its observation vector includes object, target, contact, and motion state information, allowing residual corrections to the nominal sequence as the simulated state changes.

IX. Discussion

Reach was the most reliable stage in the final evaluation. The nominal inverse-kinematics motion already provides the gross approach, while the learned residual can adjust that motion using the observed ball position and robot state. The 13.9 mm mean minimum reach error also sits well inside the 45 mm success threshold.

Grasping and retention place different demands on the controller. A hand can reach the ball closely without forming the required contact pattern after fixture release, and a grasp that initially satisfies the contact criterion can still lose the ball during lift or windup. In our setup, the policy observes contact state, ball motion, hand configuration, and arm state while operating across randomized ball radius, mass, friction, actuator gain, delay, and observation noise. The stage results show that most episodes progressed through grasp formation and that additional failures appeared during retention and release.

The release and throw stages are brief relative to the earlier phases, which motivated balancing training samples across task phases. Release requires forward ball speed above 0.35 m/s, while throw success additionally requires more than 0.16 m of forward travel and a minimum target distance below 0.50 m. These criteria make the final stage dependent on the state carried forward from reach, grasp, hold, and windup as well as the release action itself.

The comparison with the earlier hand-tuned controller supports the use of residual learning for this task. Behavioral cloning captures the scripted teacher's reach, grasp, lift, windup, and throw structure. DAgger then adds teacher labels for states visited while the learned policy increasingly controls data collection, with teacher-action probability reduced from 45% in the first round to 15% in the second. This gives the dataset examples of corrective behavior outside the original teacher-only trajectories.

X. Conclusion

We developed a residual neural controller for a sequential reach–grasp–hold–throw task using the Unitree R1 right arm and BrainCo Revo 2 Basic right hand in MuJoCo. The controller combines a nominal scripted motion with learned state-dependent corrections, and its training pipeline uses domain-randomized behavioral cloning followed by two DAgger rounds. The resulting policy was more reliable than the earlier hand-tuned controller across the reported evaluation, with reaching emerging as the strongest stage and later contact, retention, release, and targeting stages accounting for the remaining failures.

The results support the residual design for this simulated task: the nominal trajectory supplies the broad motion sequence, while the learned policy adapts it using ball, target, contact, robot-state, and previous-action information. DAgger complements the initial demonstrations by adding corrective labels at states visited by the developing policy, and phase-balanced sampling keeps the short release and throw portions represented during training.

This work is simulation-only and does not include physical-robot validation. The virtual wrist adapter is an engineering approximation and must be physically calibrated and safety-validated before sim-to-real deployment.

REFERENCES

[1] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012, pp. 5026–5033.

[2] S. Ross, G. J. Gordon, and J. A. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, vol. 15, 2011, pp. 627–635.

[3] Unitree Robotics, "unitree_mujoco: MuJoCo-based simulator for Unitree robots," GitHub repository, 2026. [Online]. Available: https://github.com/unitreerobotics/unitree_mujoco. Accessed: Oct. 6, 2026.

[4] Unitree Robotics, "unitree_rl_mjlab: Reinforcement learning implementation for Unitree robots based on MuJoCo," GitHub repository, 2026. [Online]. Available: https://github.com/unitreerobotics/unitree_rl_mjlab. Accessed: Oct. 6, 2026.

[5] BrainCo, "Revo 2 dexterous hand documentation," 2026. [Online]. Available: https://brainco.tech/docs/revolimb-hand/en/revo2/parameters.html. Accessed: Oct. 6, 2026.